

Support for Container Device Interface

About the Container Device Interface

As of the `v1.12.0` release, the NVIDIA Container Toolkit includes support for generating Container Device Interface (CDI) specifications.

CDI is an open specification for container runtimes that abstracts what access to a device, such as an NVIDIA GPU, means, and standardizes access across container runtimes. Popular container runtimes can read and process the specification to ensure that a device is available in a container. CDI simplifies adding support for devices such as NVIDIA GPUs because the specification is applicable to all container runtimes that support CDI.

CDI also improves the compatibility of the NVIDIA container stack with certain features such as rootless containers.

Generating a CDI Specification

Prerequisites

- You installed either the NVIDIA Container Toolkit or you installed the `nvidia-container-toolkit-base` package. The base package includes the container runtime and the `nvidia-ctk` command-line interface, but avoids installing the container runtime hook and transitive dependencies. The hook and dependencies are not needed on machines that use CDI exclusively.
- You installed an NVIDIA GPU Driver.

Automatic CDI Specification Generation

As of NVIDIA Container Toolkit `v1.18.0`, the CDI specification is automatically generated and updated by a systemd service called `nvidia-cdi-refresh`. This service:

- Automatically generates the CDI specification at `/var/run/cdi/nvidia.yaml` when:
 - The NVIDIA Container Toolkit is installed or upgraded
 - The NVIDIA GPU drivers are installed or upgraded
 - The system is rebooted

This ensures that the CDI specifications are up to date for the current driver and device configuration and that CDI Devices defined in these specifications are available when using native CDI support in

container engines such as Docker or Podman.

Run the following command to list the available CDI Devices:

```
nvidia-ctl cdi list
```

Known Limitations

The `nvidia-cdi-refresh` service does not currently handle the following situations:

- The removal of NVIDIA GPU drivers
- The reconfiguration of MIG devices

For these scenarios, the regeneration of CDI specifications must be manually triggered.

Customizing the Automatic CDI Refresh Service

To customize the behavior of the `nvidia-cdi-refresh` service, add environment variables to `/etc/nvidia-container-toolkit/nvidia-cdi-refresh.env`. These variables affect the behavior of the `nvidia-ctl cdi generate` command.

For example, to enable debug logging, update the configuration file as follows:

```
# /etc/nvidia-container-toolkit/nvidia-cdi-refresh.env
NVIDIA_CTK_DEBUG=1
```

For a complete list of available environment variables, run `nvidia-ctl cdi generate --help` to view the command's documentation.

! Important

Modifications to the environment file require a systemd reload and restarting the service to take effect

```
$ sudo systemctl daemon-reload
$ sudo systemctl restart nvidia-cdi-refresh.service
```

Managing the CDI Refresh Service

The `nvidia-cdi-refresh` service consists of two systemd units:

- `nvidia-cdi-refresh.path`: Monitors for changes to the system and triggers the service.

- `nvidia-cdi-refresh.service`: Generates the CDI specifications for all available devices based on the default configuration and any overrides in the environment file.

These services can be managed using standard systemd commands.

When working as expected, the `nvidia-cdi-refresh.path` service is enabled and active, and the `nvidia-cdi-refresh.service` is enabled and has run at least once. For example:

```
$ sudo systemctl status nvidia-cdi-refresh.path
• nvidia-cdi-refresh.path - Trigger CDI refresh on NVIDIA driver install / uninstall event
  Loaded: loaded (/etc/systemd/system/nvidia-cdi-refresh.path; enabled; preset: enabled)
  Active: active (waiting) since Fri 2025-06-27 06:04:54 EDT; 1h 47min ago
  Triggers: • nvidia-cdi-refresh.service
```

```
$ sudo systemctl status nvidia-cdi-refresh.service
○ nvidia-cdi-refresh.service - Refresh NVIDIA CDI specification file
  Loaded: loaded (/etc/systemd/system/nvidia-cdi-refresh.service; enabled; preset: enabled)
  Active: inactive (dead) since Fri 2025-06-27 07:17:26 EDT; 34min ago
  TriggeredBy: • nvidia-cdi-refresh.path
    Process: 1317511 ExecStart=/usr/bin/nvidia-ctl cdi generate --output=/var/run/cdi/nvidia-cdi-specs
    Main PID: 1317511 (code=exited, status=0/SUCCESS)
    CPU: 562ms
  ...
```

If these are not enabled as expected, enable them by running:

```
$ sudo systemctl enable --now nvidia-cdi-refresh.path
$ sudo systemctl enable --now nvidia-cdi-refresh.service
```

Troubleshooting CDI Specification Generation and Resolution

If CDI specifications for available devices are not generated or updated as expected, check the logs for the `nvidia-cdi-refresh.service` by running:

```
$ sudo journalctl -u nvidia-cdi-refresh.service
```

In most cases, restarting the service should be sufficient to trigger the (re)generation of CDI specifications:

```
$ sudo systemctl restart nvidia-cdi-refresh.service
```

Running:

```
$ nvidia-ctl --debug cdi list
```

Shows a list of available CDI Devices and any errors that occurred when loading CDI Specifications from `/etc/cdi` or `/var/run/cdi`.

Manual CDI Specification Generation

As of the NVIDIA Container Toolkit `v1.18.0`, the recommended mechanism to regenerate CDI specifications is to restart the `nvidia-cdi-refresh.service`:

```
$ sudo systemctl restart nvidia-cdi-refresh.service
```

If this does not work, or you need more flexibility, use the `nvidia-ctl cdi generate` command directly:

```
$ sudo nvidia-ctl cdi generate --output=/var/run/cdi/nvidia.yaml
```

JIT-CDI Mode

Just-in-time CDI (JIT-CDI) mode generates an in-memory CDI specification for the NVIDIA devices that a container requests. The NVIDIA Container Runtime uses this specification to update the container configuration. JIT-CDI mode does not write a persistent specification to `/var/run/cdi` or `/etc/cdi`.

When `nvidia-container-runtime.mode` is set to `auto`, the runtime selects JIT-CDI mode on systems that use the NVIDIA Management Library (NVML) or Windows Subsystem for Linux 2 (WSL2). Native CDI-enabled runtimes still use the persistent specifications described in [Automatic CDI Specification Generation](#).

Disabling Hooks in JIT-CDI Mode

You can prevent JIT-CDI mode from adding specific hooks to an in-memory specification. The supported hook names are `create-symlinks`, `disable-device-node-modification`, `enable-cuda-compat`, `update-application-profile`, and `update-ldcache`. Use `all` to disable every hook.

The following command disables the `update-application-profile` and `update-ldcache` hooks. Separate multiple values with a colon:

```
$ sudo nvidia-ctk config --in-place \
  --set nvidia-container-runtime.modes.jit-cdi.nvcdi-disable-hooks=update-application-p
```

The resulting configuration contains the following settings:

```
[nvidia-container-runtime.modes.jit-cdi]
nvcdi-disable-hooks = ["update-application-profile", "update-ldcache"]
```

Warning

Disable only the hook that conflicts with your environment. Hooks configure libraries, links, device behavior, and GPU visibility in the container. Disabling a required hook can prevent an application from starting or can expose more GPUs to EGL and Vulkan applications than the container requested.

For a persistent specification, pass `--disable-hook` once for each hook when you generate the CDI specification manually:

```
$ sudo nvidia-ctk cdi generate \
  --disable-hook update-application-profile \
  --disable-hook update-ldcache \
  --output=/var/run/cdi/nvidia.yaml
```

Running a Workload with CDI

Using CDI to inject NVIDIA devices can conflict with using the NVIDIA Container Runtime hook. This means that if a `/usr/share/containers/oci/hooks.d/oci-nvidia-hook.json` file exists, delete it or ensure that you do not run containers with the `NVIDIA_VISIBLE_DEVICES` environment variable set.

The use of the CDI specification depends on the CDI-enabled container engine or CLI that you use. In the case of `podman`, for example, releases as of `v4.1.0` include support for specifying CDI devices in the `--device` argument. Assuming that you generated a CDI specification as in the preceding section, running a container with access to all NVIDIA GPUs requires the following command:

```
$ podman run --rm --device nvidia.com/gpu=all --security-opt=label=disable ubuntu nvidia-
```

The preceding sample command should show the same output as running `nvidia-smi -L` on the host.

The CDI specification also contains references to individual GPUs or MIG devices. You can request these by specifying their names when launching a container, such as the following example:

```
$ podman run --rm \
  --device nvidia.com/gpu=0 \
  --device nvidia.com/gpu=1:0 \
  --security-opt=label=disable \
  ubuntu nvidia-smi -L
```

The preceding sample command requests the full GPU with index 0 and the first MIG device on GPU 1. The output should show only the UUIDs of the requested devices.

IMEX Channels

Containers can request specific NVIDIA IMEX channels with the `NVIDIA_IMEX_CHANNELS` environment variable. For supported values, an example, and validation errors, refer to [Requesting IMEX Channels](#).

Using CDI with Non-CDI-Enabled Runtimes

To support runtimes that do not natively support CDI, you can configure the NVIDIA Container Runtime in a `cdi` mode. In this mode, the NVIDIA Container Runtime does not inject the NVIDIA Container Runtime Hook into the incoming OCI runtime specification. Instead, the runtime performs the injection of the requested CDI devices.

The NVIDIA Container Runtime automatically uses `cdi` mode if you request devices by their CDI device names.

Using Docker as an example of a non-CDI-enabled runtime, the following command uses CDI to inject the requested devices into the container:

```
$ docker run --rm -ti --runtime=nvidia \
  -e NVIDIA_VISIBLE_DEVICES=nvidia.com/gpu=all \
  ubuntu nvidia-smi -L
```

The `NVIDIA_VISIBLE_DEVICES` environment variable indicates which devices to inject into the container and is explicitly set to `nvidia.com/gpu=all`.

Setting the CDI Mode Explicitly

You can force CDI mode by explicitly setting the `nvidia-container-runtime.mode` option in the NVIDIA Container Runtime config to `cdi`:

```
$ sudo nvidia-ctk config --in-place --set nvidia-container-runtime.mode=cdi
```

In this case, the `NVIDIA_VISIBLE_DEVICES` environment variable is still used to select the devices to inject into the container, but the `nvidia-container-runtime.modes.cdi.default-kind` (with a default value of `nvidia.com/gpu`) is used to construct a fully-qualified CDI device name only when you specify a device index such as `all`, `0`, or `1`, and so on.

This means that if CDI mode is explicitly enabled, the following sample command has the same effect as specifying `NVIDIA_VISIBLE_DEVICES=nvidia.com/gpu=all`.

```
$ docker run --rm -ti --runtime=nvidia \  
-e NVIDIA_VISIBLE_DEVICES=all \  
ubuntu nvidia-smi -L
```



[Privacy Policy](#) | [Your Privacy Choices](#) | [Terms of Service](#) | [Accessibility](#) | [Corporate Policies](#) | [Product Security](#) | [Contact](#)

Copyright © 2020-2026, NVIDIA Corporation.